SENTINEL-TS

Hybrid Time-Series Anomaly Intelligence for Cloud Infrastructure Telemetry

Project Synopsis · Time Series Data Analysis (TSDA) Mini Project · Dept. of Computer Engineering, SPIT

1. Project at a Glance

Field	Detail
Project name	SENTINEL-TS
Full title	Hybrid Time-Series Anomaly Intelligence for Cloud Infrastructure Telemetry
Domain	Site Reliability Engineering / IT infrastructure monitoring
Core task	Forecast expected server behaviour, then detect and classify deviations from it
Dataset	Numenta Anomaly Benchmark (NAB) — realAWSCloudwatch family
Data nature	Genuine Amazon CloudWatch production server telemetry
Scale	6 services · 25,477 observations · 5-minute sampling · ~14 days each
Ground truth	11 human-curated anomaly windows
Models compared	7 (2 baselines, 2 classical, 1 machine learning, 2 hybrid)
Deliverable	Working application: Python analysis engine + REST API + browser console

Table 1 — Project identity.

2. Dataset

2.1 Source

The dataset is the **realAWSCloudwatch** family of the **Numenta Anomaly Benchmark (NAB)**, published openly at github.com/numenta/NAB under AGPL-3.0. These are genuine Amazon CloudWatch measurements taken from production Amazon Web Services infrastructure — they are not simulated traces.

Real telemetry was chosen over a synthetic generator deliberately. Synthetic data contains exactly the structure its author put there, so a model that assumes that structure will always appear to succeed. Real server data contains flat idle stretches, reporting gaps, mixed noise regimes and anomalies of several different shapes, and it punishes assumptions that do not hold.

2.2 Structure and scale

Attribute	Value
Columns	2 — timestamp (ISO 8601 datetime, UTC), value (float)
Native sampling frequency	5 minutes (288 observations per day)
Duration per series	approximately 14 days
Observations per series	4,032 (4,730 for the two longer series)
Total observations	25,477 across 6 series
Analysis frequency	resampled to 15 minutes (96 buckets/day); user-selectable 5 min to 1 hour
Labels	anomaly windows (start, end) — not point labels

Table 2 — Dataset structure.

2.3 The six services

Series file	Metric measured	Unit	Windows
ec2_cpu_utilization_5f5533	EC2 instance CPU utilisation	%	2

Series file	Metric measured	Unit	Windows
rds_cpu_utilization_cc0c53	RDS database CPU utilisation	%	2
ec2_network_in_257a54	EC2 inbound network volume	bytes	1
elb_request_count_8c0756	Load balancer request count	req/5 min	2
ec2_disk_write_bytes_1ef3de	EC2 disk write throughput	bytes	1
grok_asg_anomaly	Auto-scaling group aggregate load	units	3

Table 3 — The six telemetry series. Each is modelled independently.

A note on usable ground truth: 11 windows exist, but at a 30% chronological hold-out only 5 of them fall inside a test set. On two services the single labelled window lies entirely within the training portion, so detection cannot be scored there at all. The evaluation therefore rests on 5 windows across 4 services, and this constrains how much confidence any ranking deserves.

3. Problem Statement

3.1 Statement

> *Cloud services emit continuous telemetry, and incidents appear in that stream before customers notice them. Static thresholds fail to detect those incidents because server load is strongly periodic: the decision rule is stationary while the process it monitors is not. The problem is to build a system that learns what each metric should be at each moment in time, flags departures from that expectation, and converts those flags into a small number of classified, actionable incidents that an on-call engineer can act on without first investigating the alert itself.*

3.2 What exactly we are solving

The failure mode is specific, well known to anyone who has carried a pager, and expensive:

- Infrastructure load is strongly diurnal and weekly. A production API may run at 70% CPU during business hours and 20% overnight.
- A threshold set at 85% to avoid overnight noise cannot see a service that silently drops from 20% to 2% at 03:00 because an upstream dependency died — a severe incident with no threshold crossing at all.
- A threshold set tight enough to catch that would fire every single morning as load legitimately ramps up.
- Teams resolve this by widening thresholds until paging stops. The alert is then still configured, still shows green on a dashboard, and detects nothing. Its effective recall is zero while appearing to provide coverage.

So the problem is **not insufficient data and not insufficient alerting**. It is that a fixed threshold cannot represent a time-varying normal. Our solution replaces the fixed threshold with a forecast: model the expected value at each timestamp, and alert on the residual between observed and expected. The decision boundary then moves with time of day automatically.

3.3 What we are trying to do — objectives

1. Build a forecasting model of normal behaviour that captures daily periodicity, slow level drift and short-memory autocorrelation, so the expected value is time-dependent rather than constant.
2. Detect anomalies as departures from that expectation, with the threshold calibrated on training data alone so the detector cannot adapt to the incidents it is meant to find.
3. Design and evaluate a hybrid model combining a linear time-series model with a nonlinear learner, and quantify what the hybridisation actually contributes rather than assuming it helps.
4. Compare seven approaches on identical chronological splits under metrics that reflect operational cost — including alert burden and detection latency, not accuracy alone.
5. Deliver a working interactive application where a user supplies input, triggers analysis and explores results — not a static notebook.
6. Convert raw detections into decision support: classified incident shapes, severity, deviation from forecast, recommended first actions, and a forward capacity outlook.

3.4 Intended users

User	What the system gives them
Site-reliability / platform engineer	Knowledge that a service is abnormal for this time of day, with enough context to begin diagnosis
On-call engineer	A small number of pages worth waking for, each with a classified incident shape and a concrete first action
Capacity planner	A forward forecast with uncertainty band and an estimate of when a service ceiling is likely to be breached
Engineering manager	Evidence that coverage is real — expressed as detection rate and alert burden, not a count of configured alerts

Table 4 — Intended users.

4. Models Used for Comparison

Seven models are fitted on identical chronological splits of the same preprocessed series. Two are deliberately simple, and they are not straw men: on strongly periodic telemetry a seasonal-naive forecast is hard to beat, and the rolling z-score is what the majority of production monitoring stacks actually ship today.

#	Model	Family	Forecasts?	Detects?
1	Seasonal Naive	Baseline	Yes	Yes (via residual)
2	Rolling Z-Score (3.5σ)	Statistical	No	Yes
3	Holt-Winters ETS	Classical	Yes	Yes (via residual)
4	SARIMAX + Fourier	Classical	Yes	Yes (via residual)
5	Isolation Forest	Machine learning	No	Yes
6	Hybrid SARIMAX + GBR	Hybrid	Yes	Yes (dual channel)
7	Hybrid SARIMAX + Neural	Hybrid (ablation)	Yes	Yes (dual channel)

Table 5 — The seven models compared.

4.1 What each model is applied for — role and case covered

Each model is in the comparison to answer a specific question. This is the mapping an examiner is most likely to probe, so it is set out explicitly.

Model	Applied for / the exact case it covers
1. Seasonal Naive	REFERENCE BENCHMARK. Predicts each value from the observation exactly one day earlier. Its in-sample error is the denominator of MASE, so it defines what 'good' means — any model with MASE ≥ 1 has not earned its complexity. Case covered: establishing the floor that every sophisticated model must beat.
2. Rolling Z-Score (3.5σ)	INDUSTRY-DEFAULT CONTROL. Trailing window supplies a robust median and MAD-scaled spread; anything beyond 3.5σ alerts. Carries no model of seasonality whatsoever. Case covered: quantifying the exact cost of ignoring time-of-day — it is the method our project argues against, included so the argument is measured rather than asserted.
3. Holt-Winters ETS	LEVEL + TREND + SEASONALITY, SMOOTHED. Additive error, damped additive trend, additive daily seasonality in a compact state vector. Case covered: series dominated by smooth level drift with a stable repeating daily profile, where an interpretable low-parameter state-space model suffices.
4. SARIMAX + Fourier	AUTOCORRELATION + LONG SEASONAL PERIOD. ARMA(p,d,q) dynamics estimated jointly with 4 Fourier harmonic pairs encoding the 24-hour cycle; order chosen by AIC search. Case covered: series with genuine short-memory autocorrelation and a daily period too long (m = 96) for a textbook seasonal ARIMA. This is also the linear stage of the hybrid.
5. Isolation Forest	MODEL-FREE, SHAPE-SENSITIVE DETECTION. Random axis-aligned splits over a causal feature window (lags, rolling moments, differences, calendar). Assumes no normality, linearity or stationarity. Case covered: multivariate-shaped oddities a univariate residual threshold misses — e.g. an ordinary value reached at an extraordinary rate. Produces no forecast, so it has no capacity-planning value.
6. Hybrid SARIMAX + GBR	**THE PRINCIPAL CONTRIBUTION. Stage 1 SARIMAX extracts the linear structure; stage 2 gradient-boosted trees learn what remains in its residuals; detection fuses a magnitude z-score with an Isolation Forest over residual shape, then debounces. Case covered: series where the linear model leaves structured residuals (confirmed here by a failed Ljung-Box test) AND where both magnitude anomalies and slow ramps must be caught.**
7. Hybrid SARIMAX + Neural	ABLATION OF THE RESIDUAL LEARNER. Identical architecture with a feed-forward network (or LSTM where PyTorch is installed) replacing gradient boosting in stage 2. Case covered: isolating whether the hybrid's advantage comes from the two-stage architecture itself or from the particular learner bolted into it. If both score alike the gain is architectural; if they diverge, the learner must be tuned per series.

Table 6 — Model-to-case mapping. This is the central design rationale of the comparison.

5. The Hybrid Model in Detail

The premise, following Zhang (2003), is that a series decomposes into a linear component a well-specified ARMA can capture and a nonlinear component it structurally cannot. Fitting one model to both forces a compromise; fitting them in sequence lets each do what it is good at.

Stage	Component	Purpose and what it specifically handles
1	SARIMAX + Fourier	Extracts autocorrelation and the daily cycle. Produces the linear forecast $\hat{L}_t$. The daily period m = 96 is encoded as 4 Fourier harmonic pairs (8 coefficients) instead of 96 seasonal lags, which is what makes the model estimable from 941 training observations.
2	Gradient-boosted trees	Learns the residual $e_t = y_t - \hat{L}_t$ from its own 12 preceding lags plus calendar features. Captures burst clustering, asymmetric recovery after a spike, and weekday effects that are not pure harmonics. Removed 70.3% of the variance the linear model left behind.
3a	Robust z-score channel	Magnitude detection on the hybrid residual, calibrated on training residuals only. Weight 0.70. Sharp on size but blind to shape.
3b	Isolation Forest channel	Shape detection over residual behaviour — local volatility, momentum, persistence, slope. Weight 0.30, capped at 6σ-equivalent so it can tip a borderline reading over the line but never raise an alert alone. Catches slow ramps that never exceed σ.
4	Debounce	Requires 2 consecutive buckets above threshold. Converts a large share of false alarms into nothing, at the cost of one bucket of latency.

Table 7 — Hybrid architecture, stage by stage.

5.1 Why each stage exists

No stage was added speculatively. Each responds to a measured deficiency of the one before it, and this is the strongest defence of the design:

Evidence found	Stage it forced
Ljung-Box test at lag 24 gives p ≈ 0.000 — residual autocorrelation remains after the linear fit	Stage 2 (nonlinear residual learner)
A magnitude threshold alone cannot see a slow ramp that never exceeds σ	Stage 3b (residual-shape channel)
Single-bucket exceedances are overwhelmingly noise	Stage 4 (debounce)
Unclamped stage 2 explained away sustained anomalies and went silent mid-incident (0/2 windows on one service)	2σ clamp on the correction
Unbounded shape channel flagged 42% of a hold-out on a very regular training window	6σ cap on the shape channel

Table 8 — Every design element traced to the evidence that demanded it.

6. How the Comparison is Judged

Metric	What it answers	Why it is in the comparison
MASE	Did it beat the seasonal-naive benchmark?	Scale-free; MAPE is undefined at zero and byte counts legitimately idle at zero
RMSE / MAE	How far off is the forecast?	Standard forecast accuracy, reported per-service only (units differ across services)
Point-adjusted F1	Was each incident caught at all?	The headline — one page is enough to engage a human
Point-wise F1	Did it agree on every bucket?	Guards against a detector that wins by firing constantly
Segment recall	What fraction of windows were recovered?	The coverage question a manager actually asks
Detection latency	How long until the first alert?	A perfect but slow detector is still a bad pager
False alarms / day	How often does it page for nothing?	Decides whether the alert stays switched on at all

Table 9 — Evaluation metrics and their justification.

A single composite operational score orders the leaderboard: point-adjusted F1 at 0.45, segment recall 0.25, point-wise F1 0.15, detection latency 0.10 and alert burden 0.05. Alert burden carries negative weight so that no model can win by flagging everything.

6.1 Headline results

Model	Macro comp.	PA F1	Windows	FA/day
Rolling Z-Score (3.5σ)	**0.787**	**0.901**	**5 / 5**	**2.52**
Holt-Winters ETS	0.749	0.823	5 / 5	4.10
SARIMAX + Fourier	0.709	0.708	5 / 5	1.86
Hybrid SARIMAX + GBR	**0.707**	**0.687**	**5 / 5**	**1.36**
Seasonal Naive	0.697	0.760	5 / 5	4.87
Isolation Forest	0.552	0.655	3 / 5	1.06
Hybrid SARIMAX + Neural	0.532	0.456	4 / 5	0.92

Table 10 — Fleet benchmark across the labelled corpus.

The result is reported as found. The simple rolling z-score leads on the composite score. Three honest reasons: the composite weights are ours and, once every model recovers all five windows, the ranking turns largely on point-wise agreement across wide NAB windows; the evidence base is five windows across four services, too thin to separate models differing by 0.08; and on the series examined in detail the daily cycle is weak, so a trailing-window detector tracks the drifting level perfectly well. What should be added in its defence: the hybrid is **the quietest detector that still recovers every labelled window** — 1.36 false alarms per day against the baseline's 2.52 — and it wins outright on two of six services. Under an alert-burden-sensitive weighting it is preferable.

On the single-service run (EC2 CPU utilisation), the Hybrid SARIMAX + GBR ranks first at a composite of 0.860 with zero false alarms per day and all forecasters beating the seasonal-naive benchmark decisively on MASE (0.44–0.48 against 1.30).

6.2 What the system produces for the user

- A selected model per service, chosen by running the full comparison rather than fixed in advance.
- Incidents merged from contiguous flags, each with severity, classified shape (spike, dip, level shift, ramp, volatility change), duration, peak value, expected value, and percentage deviation from forecast.
- A recommended first action per incident shape — for a ramp, inspect memory and connection pools; for a level shift, check the deploy timeline; for a volatility change, look for contention rather than load.

- A forward capacity outlook with a 95% prediction interval, reporting the first bucket whose upper bound crosses the service ceiling and how many minutes away that is.

7. One-Paragraph Summary

> SENTINEL-TS is a time-series anomaly detection system for cloud server telemetry, built on the Numenta Anomaly Benchmark's realAWSCloudwatch corpus of genuine Amazon CloudWatch data. It solves the problem that static monitoring thresholds cannot represent a time-varying normal: server load is strongly periodic, so any fixed ceiling is either blind at the daily trough or deafening at the peak. The system instead forecasts what each metric should be at each moment and alerts on the residual. Seven models are compared on identical chronological splits — a seasonal-naive benchmark, an industry-default rolling z-score, Holt-Winters ETS, SARIMAX with Fourier seasonal regressors, an Isolation Forest, and two variants of a two-stage hybrid that corrects the SARIMAX forecast with a nonlinear residual learner before fusing magnitude and shape detection channels. The nonlinear stage removes 70.3% of the variance the linear model leaves behind. Detections are converted into classified incidents with severity, deviation from forecast, and recommended operator actions, delivered through a working application with a Python analysis engine, a REST API and a browser console.

Key numbers

Quantity	Value
Raw observations per series	4,032 (5-minute sampling, ~14 days)
After resampling to 15 minutes	1,345
Train / test split	941 / 404 (30% chronological hold-out)
Seasonal period	m = 96 buckets per day
Fourier harmonics	K = 4 → 8 coefficients (vs 96 seasonal lags)
Selected SARIMAX order	(3, 1, 3), AIC 2,616.99
Stationarity verdict	Non-stationary (ADF p = 0.897, KPSS p ≤ 0.01) → d = 1
Residual variance removed by stage 2	70.3%
Best single-service model	Hybrid SARIMAX + GBR, composite 0.860
Best fleet model	Rolling Z-Score, macro composite 0.787
Labelled windows evaluated	5 of 11 (the rest fall inside training)
Test coverage	101 Python tests + 27 console checks

Table 11 — Figures to have at hand.